se compilar sim mano

1 Template

```cpp
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;
const ll INF = LLONG_MAX / 4;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
}
```

2 Busca Binária

Quando usar: existe um predicado monótono `ok(x)` (F...F V...V). São $O(\log R)$ chamadas de `ok`.

```cpp
// menor x em [lo, hi] com ok(x) == true
// (ok(hi) precisa ser true)
ll lo = L, hi = R;
while (lo < hi) {
    ll mid = lo + (hi - lo) / 2;
    if (ok(mid)) hi = mid;
    else lo = mid + 1;
}
// resposta: lo

// maior x com ok(x) true (V...V F...F)
while (lo < hi) {
    ll mid = lo + (hi - lo + 1) / 2;
    if (ok(mid)) lo = mid;
    else hi = mid - 1;
}

// valores reais: número fixo de iterações
double a = 0, b = 1e9;
for (int it = 0; it < 100; it++) {
    double m = (a + b) / 2;
    if (ok(m)) b = m; else a = m;
}
```

STL em vetores ordenados: `lower_bound` devolve o primeiro $\geq x$ e `upper_bound` o primeiro $> x$. A quantidade de x é `upper - lower`.

3 Dois Ponteiros / Janela Deslizante

Quando usar: as duas pontas só andam para frente. $O(n)$.

```cpp
// maior subarray com soma <= S (a[i] >= 0)
int best = 0; ll sum = 0;
for (int l = 0, r = 0; r < n; r++) {
    sum += a[r];
    while (sum > S) sum -= a[l++];
    best = max(best, r - l + 1);
}

// par com soma X em um vetor ordenado
int i = 0, j = n - 1;
while (i < j) {
    ll s = a[i] + a[j];
    if (s == X) break;          // achou
    if (s < X) i++; else j--;
}

// mínimo de cada janela de tamanho k
// (deque monótono de índices)
deque<int> dq;
for (int i = 0; i < n; i++) {
    while (!dq.empty() && a[dq.back()] >= a[i])
        dq.pop_back();
    dq.push_back(i);
    if (dq.front() <= i - k) dq.pop_front();
    if (i >= k - 1) mn[i - k + 1] = a[dq.front()];
}
```

Para o máximo da janela, troque `>=` por `<=`.

4 Grafos

4.1 BFS — menor caminho sem pesos, $O(V+E)$

```cpp
vector<vector<int>> g;

vector<int> bfs(int s) {
    vector<int> d(g.size(), -1);
    queue<int> q; q.push(s); d[s] = 0;
    while (!q.empty()) {
        int u = q.front(); q.pop();
        for (int v : g[u]) if (d[v] == -1) {
            d[v] = d[u] + 1;
            q.push(v);
        }
    }
    return d;
}
```

4.2 BFS em grid

```cpp
int dx[] = {1, -1, 0, 0}, dy[] = {0, 0, 1, -1};
// d[R][C] preenchido com -1, d[sx][sy] = 0
queue<array<int,2>> q; q.push({sx, sy});
while (!q.empty()) {
    auto [x, y] = q.front(); q.pop();
    for (int k = 0; k < 4; k++) {
        int nx = x + dx[k], ny = y + dy[k];
        if (nx < 0 || ny < 0 || nx >= R || ny >= C)
            continue;
        if (grid[nx][ny] == '#' || d[nx][ny] != -1)
            continue;
        d[nx][ny] = d[x][y] + 1;
        q.push({nx, ny});
    }
}
```

BFS 0-1: com um `deque`, arestas de peso 0 entram na frente e arestas de peso 1 entram no fim.

4.3 DFS — componentes conexas

```cpp
vector<int> comp;   // tamanho n, preenchido com -1

void dfs(int u, int c) {
    comp[u] = c;
    for (int v : g[u])
        if (comp[v] == -1) dfs(v, c);
}

int cnt = 0;
for (int i = 0; i < n; i++)
    if (comp[i] == -1) dfs(i, cnt++);
```

A recursão pode chegar a profundidade n. Para grafos muito profundos ($\sim 10^6$), use uma pilha explícita.

4.4 Grafo bipartido (2-coloração)

```cpp
bool bipartite(int n) {
    vector<int> col(n, -1);
    for (int s = 0; s < n; s++) {
        if (col[s] != -1) continue;
        queue<int> q; q.push(s); col[s] = 0;
        while (!q.empty()) {
            int u = q.front(); q.pop();
            for (int v : g[u]) {
                if (col[v] == -1) {
                    col[v] = col[u] ^ 1; q.push(v);
                } else if (col[v] == col[u])
                    return false;
            }
        }
    }
    return true;
}
```

4.5 Encontrar um ciclo (grafo não direcionado) — $O(V+E)$

```cpp
const int MAXN = 2e5;
vector<int> adj[MAXN];
bool vis[MAXN];
vector<int> cycle;

// -1: nenhum ciclo encontrado
// -2: ciclo encontrado e já completamente reconstruído
// >= 0: vértice onde o ciclo começou

int dfs(int u, int p) {
    if (vis[u]) return u;
    vis[u] = true;

    for (int v : adj[u]) if (v != p) {
        int ret = dfs(v, u);

        if (ret == -2) return -2;
        if (ret != -1) {
            cycle.push_back(u);
            if (u == ret) return -2;
            else return ret;
        }
    }

    return -1;
}
```

Chame `dfs(s, -1)` para cada `s` não visitado até devolver `-2`; os vértices do ciclo ficam em `cycle`. Arestas múltiplas entre o mesmo par não contam como ciclo.

4.6 Ordenação topológica (Kahn) — $O(V+E)$

```
const int MAXN = 2e5 + 5;
vector<int> adj[MAXN];
int indeg[MAXN];
int n;

vector<int> toposort() {
    memset(indeg, 0, sizeof(indeg));
    vector<int> ord;

    for (int u = 1; u <= n; u++)
        for (int v : adj[u])
            indeg[v]++;

    queue<int> q;
    for (int u = 1; u <= n; u++)
        if (indeg[u] == 0)
            q.push(u);

    while (!q.empty()) {
        int u = q.front(); q.pop();
        ord.push_back(u);
        for (int v : adj[u]) {
            indeg[v]--;
            if (indeg[v] == 0) q.push(v);
        }
    }

    // Se ord.size() < n, existe um ciclo e não há ordenação topológica
    return ord;
}
```

Vértices de 1 a `n`. Para a menor ordem lexicográfica, troque a `queue` por `priority_queue<int, vector<int>, greater<int>>` ($O((V+E)\log V)$).

4.7 Dijkstra — $O((V+E)\log V)$

Quando usar: pesos não negativos.

```
vector<vector<array<ll,2>>> adj;  // {destino, peso}

vector<ll> dijkstra(int s, vector<int>& par) {
    int n = adj.size();
    vector<ll> d(n, INF);
    par.assign(n, -1);
    priority_queue<array<ll,2>,
        vector<array<ll,2>>, greater<>> pq;  // {dist, v}
    d[s] = 0; pq.push({0, s});
    while (!pq.empty()) {
        auto [du, u] = pq.top(); pq.pop();
        if (du != d[u]) continue;   // entrada velha
        for (auto [v, w] : adj[u])
            if (d[u] + w < d[v]) {
                d[v] = d[u] + w;
                par[v] = u;
                pq.push({d[v], v});
            }
    }
    return d;
}

// caminho s -> t (se d[t] < INF)
vector<int> path;
for (int v = t; v != -1; v = par[v])
    path.push_back(v);
reverse(path.begin(), path.end());
```

4.8 Dijkstra denso — $O(V^2+E)$

Quando usar: grafo denso ($E \approx V^2$, $V \lesssim 5000$), onde ganha da versão com heap. Mesmo `adj` de cima.

```
vector<ll> dijkstraDenso(int s) {
    int n = adj.size();
    vector<ll> d(n, INF);
    vector<bool> vis(n, false);
    d[s] = 0;
    for (int i = 0; i < n; i++) {  // 1 vértice por vez
        int cur = -1;              // não visitado mais perto
        for (int j = 0; j < n; j++)
            if (!vis[j] && (cur == -1 || d[j] < d[cur]))
                cur = j;
        if (d[cur] == INF) break;  // resto inalcançável
        vis[cur] = true;
        for (auto [v, w] : adj[cur])
            d[v] = min(d[v], d[cur] + w);
    }
    return d;
}
```

4.9 Bellman–Ford — pesos negativos, $O(VE)$

```
// arestas {u, v, w}
// false se há ciclo negativo alcançável a partir de s
bool bellmanFord(int n, int s,
                 const vector<array<ll,3>>& E,
                 vector<ll>& d) {
    d.assign(n, INF); d[s] = 0;
    for (int rep = 0; rep < n - 1; rep++)
        for (auto [u, v, w] : E)
            if (d[u] < INF) d[v] = min(d[v], d[u] + w);
    // ciclo negativo: alguém ainda melhora
    for (auto [u, v, w] : E)
        if (d[u] < INF && d[u] + w < d[v]) return false;
    return true;
}
```

O `d[u] < INF` evita "melhorar" a partir de vértices inalcançáveis quando há pesos negativos.

4.10 Floyd–Warshall — todos os pares, $O(n^3)$

```
// d[i][j] = peso da aresta, 0 na diagonal,
// INF se não há aresta
for (int k = 0; k < n; k++)
  for (int i = 0; i < n; i++)
    for (int j = 0; j < n; j++)
      if (d[i][k] < INF && d[k][j] < INF)
        d[i][j] = min(d[i][j], d[i][k] + d[k][j]);
// i está num ciclo negativo sse d[i][i] < 0
```

Funciona para $n \lesssim 500$. O laço de `k` tem que ser o de fora.

4.11 Union-Find (DSU)

Compressão de caminho mais união por tamanho dá $\approx O(\alpha(n))$ por operação. Tamanho `n + 1`, então serve para vértices de 0 a `n-1` ou de 1 a `n`.

```
struct DSU {
    vector<int> p, sz;
    DSU(int n) : p(n + 1), sz(n + 1, 1) {
        iota(p.begin(), p.end(), 0);
    }
    int find(int x) {
        return p[x] == x ? x : p[x] = find(p[x]);
    }
    // false se a e b já estavam juntos
    bool unite(int a, int b) {
        a = find(a); b = find(b);
        if (a == b) return false;
        if (sz[a] < sz[b]) swap(a, b);
        p[b] = a; sz[a] += sz[b];
        return true;
    }
    bool same(int a, int b) {
        return find(a) == find(b);
    }
};
```

4.12 Kruskal (árvore geradora mínima) — $O(E\log E)$

```
// arestas {custo, u, v}: o custo vem primeiro
// para o sort ordenar por ele
// devolve o custo da MST, ou -1 se o grafo é desconexo
ll kruskal(int n, vector<array<ll,3>> arestas) {
    sort(arestas.begin(), arestas.end());
    DSU dsu(n);
    ll custo = 0; int usadas = 0;
    for (auto [c, u, v] : arestas)
        if (dsu.unite(u, v)) { custo += c; usadas++; }
    return usadas == n - 1 ? custo : -1;
}
```

Para a árvore geradora **máxima**, ordene ao contrário: `sort(arestas.rbegin(), arestas.rend())`.

5 Grafos Avançados

5.1 Johnson — todos os pares com pesos negativos, $O(VE + V(V+E)\log V)$

Quando usar: pesos negativos e grafo esparso grande demais para o Floyd–Warshall ($V \gtrsim 500$).

Ideia: um Bellman–Ford acha potenciais `h` que deixam todo peso $w + h_u - h_v \geq 0$. Aí roda um Dijkstra de cada vértice e desfaz a troca no fim.

```
// arestas {u, v, w}, vértices 0..n-1
// false se há ciclo negativo
bool johnson(int n, const vector<array<ll,3>>& E,
             vector<vector<ll>>& dist) {
    // Bellman-Ford de um vértice virtual ligado a
    // todos com peso 0 = começar com h = 0 em todos
    vector<ll> h(n, 0);
    for (int rep = 0; rep < n; rep++)
        for (auto [u, v, w] : E)
            h[v] = min(h[v], h[u] + w);
    for (auto [u, v, w] : E)
        if (h[u] + w < h[v]) return false;

    // pesos novos, todos >= 0
    vector<vector<array<ll,2>>> g(n);
    for (auto [u, v, w] : E)
        g[u].push_back({v, w + h[u] - h[v]});

    dist.assign(n, vector<ll>(n, INF));
    for (int s = 0; s < n; s++) {       // Dijkstra
        auto& d = dist[s];
        priority_queue<array<ll,2>,
            vector<array<ll,2>>, greater<>> pq;
        d[s] = 0; pq.push({0, s});
        while (!pq.empty()) {
            auto [du, u] = pq.top(); pq.pop();
            if (du != d[u]) continue;
            for (auto [v, w] : g[u])
                if (du + w < d[v]) {
                    d[v] = du + w;
                    pq.push({d[v], v});
                }
        }
        for (int v = 0; v < n; v++)    // desfaz a troca
            if (d[v] < INF) d[v] += h[v] - h[s];
    }
    return true;
}
```

5.2 Karp — ciclo de média mínima, $O(VE)$

Retorna: o menor valor de $\frac{\text{peso do ciclo}}{\text{nº de arestas}}$ entre todos os ciclos de um grafo **direcionado**.

Ideia: com `D[k][v]` = menor peso de um passeio de exatamente k arestas que termina em v (começando em qualquer vértice), a resposta é

$$\mu = \min_{v} \max_{0 \leq k < n} \frac{D_n(v) - D_k(v)}{n - k}.$$

```
// arestas {u, v, w}, vértices 0..n-1
// false se o grafo não tem ciclo
bool karp(int n, const vector<array<ll,3>>& E,
          double& mu) {
    vector<vector<ll>> D(n + 1, vector<ll>(n, INF));
    D[0].assign(n, 0);          // começa em qualquer um
    for (int k = 1; k <= n; k++)
        for (auto [u, v, w] : E)
            if (D[k-1][u] < INF)
                D[k][v] = min(D[k][v], D[k-1][u] + w);

    bool achou = false; mu = 1e18;
    for (int v = 0; v < n; v++) {
        if (D[n][v] >= INF) continue;
        double pior = -1e18;
        for (int k = 0; k < n; k++)
            if (D[k][v] < INF)
                pior = max(pior,
                    (double)(D[n][v] - D[k][v]) / (n - k));
        mu = min(mu, pior);
        achou = true;
    }
    return achou;
}
```

Usa $O(V^2)$ de memória, então vale até $V \approx 3000$. Existe ciclo negativo sse $\mu < 0$. Para o ciclo de média **máxima**, troque o sinal de todos os pesos e da resposta.

5.3 Pontes e pontos de articulação — $O(V+E)$

Retorna: as **pontes** (arestas cuja remoção desconecta o grafo) e os **pontos de articulação** (vértices cuja remoção desconecta o grafo). Grafo não direcionado.

```
// adj[u] = {v, id da aresta}; ao ler a aresta j:
// adj[u].push_back({v, j}); adj[v].push_back({u, j});
vector<vector<array<int,2>>> adj;
vector<int> tin, low;
vector<bool> corte;           // ponto de articulação?
vector<array<int,2>> pontes;  // {u, v}
int timer_;

void dfs(int u, int idPai) {
    tin[u] = low[u] = timer_++;
    int filhos = 0;
    for (auto [v, id] : adj[u]) {
        if (id == idPai) continue;  // não volta pela mesma aresta
        if (tin[v] != -1) {         // aresta de retorno
            low[u] = min(low[u], tin[v]);
        } else {
            dfs(v, id);
            low[u] = min(low[u], low[v]);
            filhos++;
            if (low[v] > tin[u]) pontes.push_back({u, v});
            if (low[v] >= tin[u] && idPai != -1)
                corte[u] = true;
        }
    }
    if (idPai == -1 && filhos > 1) corte[u] = true;  // raiz
}

void acharPontes(int n) {
    tin.assign(n, -1); low.assign(n, -1);
    corte.assign(n, false); pontes.clear(); timer_ = 0;
    for (int i = 0; i < n; i++)
        if (tin[i] == -1) dfs(i, -1);
}
```

`tin[u]` é a ordem em que a DFS chegou em `u`, e `low[u]` é o menor `tin` que a subárvore de `u` alcança usando uma aresta de retorno. Se a subárvore de `v` não consegue subir acima de `u` (`low[v] > tin[u]`), a aresta u–v é ponte. Se não consegue passar de `u` (`low[v] >= tin[u]`), tirar `u` isola `v`. A raiz é exceção: é articulação só se tiver 2 ou mais filhos na DFS.

Pular pelo **id** da aresta, e não pelo vértice pai, faz arestas paralelas funcionarem: duas arestas entre u e v nunca são ponte.

6 Backtracking

Molde: escolhe → recursão → desfaz. O estado fica global e cada chamada desfaz exatamente o que fez. Limites práticos: 2^n com $n \leq 20$–25 e $n!$ com $n \leq 10$–11.

6.1 Subconjuntos — $O(2^n)$

```
int n; vector<int> a, cur;

void subsets(int i) {
    if (i == n) { /* usa cur */ return; }
    cur.push_back(a[i]); subsets(i + 1);  // pega a[i]
    cur.pop_back();      subsets(i + 1);  // não pega
}

// versão iterativa: o bit i de mask diz se pega a[i]
for (int mask = 0; mask < (1 << n); mask++)
    for (int i = 0; i < n; i++)
        if (mask >> i & 1) { /* a[i] está dentro */ }
```

6.2 Combinações (k de n) — $O\left(\binom{n}{k} \cdot k\right)$

```
// escolhe k elementos de a[i..n-1], em ordem
void comb(int i, int k) {
    if (k == 0) { /* usa cur */ return; }
    for (int j = i; j <= n - k; j++) {  // sobram >= k
        // com a ordenado, pula repetidos:
        // if (j > i && a[j] == a[j-1]) continue;
        cur.push_back(a[j]);
        comb(j + 1, k - 1);
        cur.pop_back();
    }
}
```

6.3 Permutações — $O(n! \cdot n)$

```
vector<bool> usado;   // tamanho n

void perm() {
    if ((int)cur.size() == n) { /* usa cur */ return; }
    for (int j = 0; j < n; j++) if (!usado[j]) {
        usado[j] = true;  cur.push_back(a[j]);
        perm();
        cur.pop_back();   usado[j] = false;
    }
}
```

Sem poda nenhuma, `next_permutation` (seção STL) é mais curto. A versão recursiva vale quando dá para cortar prefixos inválidos cedo.

6.4 Poda (branch and bound)

```
ll melhor = INF;

void bt(int i, ll custo /*, estado */) {
    if (custo >= melhor) return;  // poda: não melhora
    if (/* estado inválido */) return;  // poda: inviável
    if (i == n) { melhor = custo; return; }
    for (/* cada escolha c */) {
        // aplica c no estado
        bt(i + 1, custo + /* custo de c */);
        // desfaz c
    }
}
```

Cortar um ramo **antes** de explorá-lo é o que torna o backtracking viável. Fica mais forte com: um limite otimista (`custo + estimativa do resto >= melhor`), testar primeiro as escolhas mais promissoras (ordenar), e checar inviabilidade o mais cedo possível.

6.5 Estado em bitmask

```
// usados = máscara das opções já escolhidas
void bt(int usados) {
    int cheio = (1 << n) - 1;
    if (usados == cheio) { /* chegou ao fim */ return; }
    int livres = ~usados & cheio;  // 0 = proibidos
    for (int r = livres; r; r &= r - 1) {
        int j = __builtin_ctz(r);  // próxima livre
        bt(usados | 1 << j);
    }
}
```

O estado vai **por valor**, então não precisa desfazer nada. Percorrer só os bits livres evita testar opções que já se sabe que não servem.

6.6 Máscara em base K (K estados por item)

Cada item i vira um **dígito** de 0 a $K-1$ (ex.: $K=3$: fora / grupo A / grupo B), e o estado inteiro é um número na base K, de 0 a $K^n - 1$.

```
int K, pw[20];  // pw[i] = K^i
// pw[0] = 1; for (i = 1..n) pw[i] = pw[i-1] * K;

// lê o dígito i            (como m >> i & 1)
int dig(int m, int i) { return m / pw[i] % K; }

// troca o dígito i por d   (como ligar/desligar bit)
int setd(int m, int i, int d) {
    return m + (d - dig(m, i)) * pw[i];
}

for (int m = 0; m < pw[n]; m++)      // todos os K^n
    for (int i = 0; i < n; i++) {
        int d = dig(m, i);           // estado do item i
    }
```

Serve de índice em DP: `dp[m][v]` com o dígito i = quantas vezes a cidade i foi visitada (0, 1 ou 2). Cuidado com o tamanho: $3^{13} \approx 1{,}6 \cdot 10^6$ e $3^{15} \approx 1{,}4 \cdot 10^7$. Se K é potência de 2, use bits direto: com $K=4$, o item i é `m >> 2*i & 3`.

6.7 Máscaras por nº de bits (truque de Gosper)

Exatamente k bits 1, em ordem crescente, $O\left(\binom{n}{k}\right)$:

```
// k >= 1 e n <= 30
for (int m = (1 << k) - 1; m < (1 << n); ) {
    // usa m
    int c = m & -m, r = m + c;
    m = (((r ^ m) >> 2) / c) | r;   // próxima com k bits
}
// n = 3, k = 2:  011, 101, 110
```

Todas, agrupadas pelo nº de bits ($000 \mid 001,010,100 \mid 011,101,110 \mid 111$):

```
vector<int> ms(1 << n);
iota(ms.begin(), ms.end(), 0);
stable_sort(ms.begin(), ms.end(), [](int a, int b) {
    return __builtin_popcount(a) < __builtin_popcount(b);
});
for (int m : ms) { /* usa m */ }
```

Útil para escolher exatamente k itens e para DP em camadas, em que o estado com j itens só depende dos estados com $j-1$.

6.8 Marca e desmarca (grid)

```
int dx[] = {1, -1, 0, 0}, dy[] = {0, 0, 1, -1};
vector<vector<bool>> vis;   // R x C

void bt(int x, int y /*, estado */) {
    if (x < 0 || y < 0 || x >= R || y >= C) return;
    if (vis[x][y] /* || bloqueado */) return;
    vis[x][y] = true;                       // marca
    for (int d = 0; d < 4; d++)
        bt(x + dx[d], y + dy[d]);
    vis[x][y] = false;                      // desmarca
}
```

Diferente da DFS comum, desmarcar na volta deixa a célula livre para **outros caminhos**: assim se exploram todos os caminhos simples, não só os vértices alcançáveis.

6.9 Meet in the middle — $O(2^{n/2} \cdot n)$

Quando 2^n é demais ($n \le 40$): divide em duas metades, gera todas as somas de cada uma e combina com busca binária.

```
// todas as somas de subconjuntos de v
vector<ll> somas(const vector<ll>& v) {
    vector<ll> r = {0};
    for (ll x : v) {
        int k = r.size();
        for (int i = 0; i < k; i++) r.push_back(r[i] + x);
    }
    return r;
}

int h = a.size() / 2;
auto esq = somas({a.begin(), a.begin() + h});
auto dir = somas({a.begin() + h, a.end()});
sort(dir.begin(), dir.end());
for (ll x : esq) {
    // procura em dir o complemento de x com
    // lower_bound / upper_bound
}
```

Se os mesmos estados se repetem na recursão, guarde os resultados (memoização) e o backtracking vira DP.

7 Programação Dinâmica Clássica

7.1 Mochila 0/1 — $O(nW)$

```
vector<ll> dp(W + 1, 0);
for (int i = 0; i < n; i++)
    for (int c = W; c >= wt[i]; c--)
        dp[c] = max(dp[c], dp[c - wt[i]] + val[i]);
// resposta: dp[W]
```

O laço vai de trás para frente para que cada item seja usado no máximo uma vez. Na versão **ilimitada** (itens reutilizáveis), percorra `c` para frente.

7.2 LIS — $O(n \log n)$

```
// t[k] = menor final possível de uma subsequência
//        crescente de tamanho k + 1
vector<int> t;
for (int x : a) {
    auto it = lower_bound(t.begin(), t.end(), x);
    if (it == t.end()) t.push_back(x);
    else *it = x;
}
// tamanho da LIS = t.size()
```

Use `upper_bound` para a maior subsequência **não decrescente**.

7.3 LCS — $O(nm)$

```
int n = s.size(), m = t.size();
vector<vector<int>> dp(n + 1,
                       vector<int>(m + 1, 0));
for (int i = 1; i <= n; i++)
    for (int j = 1; j <= m; j++)
        dp[i][j] = (s[i-1] == t[j-1])
            ? dp[i-1][j-1] + 1
            : max(dp[i-1][j], dp[i][j-1]);
// resposta: dp[n][m]
```

Para reconstruir, volte a partir de (n,m): se os caracteres batem, ande na diagonal; senão, vá para o vizinho maior. A **distância de edição** usa a mesma tabela, com $dp[i][j] = \min(dp[i-1][j]+1,\ dp[i][j-1]+1,\ dp[i-1][j-1]+[s_i \neq t_j])$, onde $dp[i][0] = i$ e $dp[0][j] = j$.

7.4 Troco de Moedas — $O(nX)$

```
// menor número de moedas para formar cada valor
vector<ll> mn(X + 1, INF); mn[0] = 0;
for (int x = 1; x <= X; x++)
    for (int c : coins)
        if (c <= x && mn[x - c] < INF)
            mn[x] = min(mn[x], mn[x - c] + 1);
// mn[X] == INF significa impossível

// número de maneiras (ordem não importa)
vector<ll> ways(X + 1, 0); ways[0] = 1;
for (int c : coins)              // moedas por fora!
    for (int x = c; x <= X; x++)
        ways[x] = (ways[x] + ways[x - c]) % MOD;
```

Com as moedas no laço de fora, cada multiconjunto é contado uma vez. Trocando os laços (`x` por fora), contam-se sequências **ordenadas**, então $1+2$ e $2+1$ viram casos diferentes.

7.5 DP com Bitmask (Caixeiro Viajante) — $O(2^n n^2)$

`dp[mask][v]` é o custo mínimo para sair de 0, visitar exatamente os vértices de `mask` e terminar em `v`.

```
int FULL = 1 << n;
vector<vector<ll>> dp(FULL, vector<ll>(n, INF));
dp[1][0] = 0;
for (int mask = 1; mask < FULL; mask++)
    for (int v = 0; v < n; v++) {
        if (!(mask >> v & 1) || dp[mask][v] >= INF)
            continue;
        for (int u = 0; u < n; u++) {
            if (mask >> u & 1) continue;
            int nm = mask | (1 << u);
            dp[nm][u] = min(dp[nm][u],
                            dp[mask][v] + w[v][u]);
        }
    }
ll ans = INF;   // ciclo fechado voltando para 0
for (int v = 1; v < n; v++)
    ans = min(ans, dp[FULL - 1][v] + w[v][0]);

// percorrer todas as submáscaras s de mask
for (int s = mask; s > 0; s = (s - 1) & mask) {}
```

O limite prático é $n \leq 18$–20. Cuidado com a memória: $2^{20} \cdot 20$ valores `ll` dão uns 160 MB, então use `int` se os custos couberem.

8 Aritmética Modular

```
const ll MOD = 1e9 + 7;

ll power(ll b, ll e, ll m = MOD) {
    ll r = 1; b %= m; if (b < 0) b += m;
    while (e > 0) {
        if (e & 1) r = r * b % m;
        b = b * b % m;
        e >>= 1;
    }
    return r;
}

// inverso quando MOD é primo (Fermat)
ll inv(ll a) { return power(a, MOD - 2); }

// a*x + b*y = gcd(a, b)
ll egcd(ll a, ll b, ll& x, ll& y) {
    if (b == 0) { x = 1; y = 0; return a; }
    ll x1, y1, g = egcd(b, a % b, x1, y1);
    x = y1;
    y = x1 - (a / b) * y1;
    return g;
}
// inverso módulo qualquer m com gcd(a, m) = 1:
// egcd(a, m, x, y); resposta (x % m + m) % m

// coeficientes binomiais
const int MAXN = 1e6 + 5;
ll fact[MAXN], ifact[MAXN];
void initFact() {
    fact[0] = 1;
    for (int i = 1; i < MAXN; i++)
        fact[i] = fact[i-1] * i % MOD;
    ifact[MAXN-1] = inv(fact[MAXN-1]);
    for (int i = MAXN - 1; i > 0; i--)
        ifact[i-1] = ifact[i] * i % MOD;
}
ll C(int n, int k) {
    if (k < 0 || k > n) return 0;
    return fact[n] * ifact[k] % MOD
                   * ifact[n-k] % MOD;
}
```

Para subtrair, use `(a - b + MOD) % MOD`. Use `std::gcd` e `lcm = a / gcd(a,b) * b` (divida antes para evitar overflow).

9 Algoritmos de String

9.1 KMP (função prefixo) — $O(n)$

Retorna: `pi[i]` = tamanho do maior prefixo próprio de `s[0..i]` que também é sufixo dele. Ex.: `"aabaaab"` → `[0,1,0,1,2,2,3]`.

```
vector<int> prefixFunction(const string& s) {
    int n = s.size();
    vector<int> pi(n, 0);
    for (int i = 1; i < n; i++) {
        int j = pi[i-1];
        while (j > 0 && s[i] != s[j]) j = pi[j-1];
        if (s[i] == s[j]) j++;
        pi[i] = j;
    }
    return pi;
}

// todas as ocorrências de p em t
string c = p + '#' + t;
auto pi = prefixFunction(c);
int m = p.size();
for (int i = 0; i < (int)c.size(); i++)
    if (pi[i] == m) ans.push_back(i - 2 * m);
```

9.2 Função Z — $O(n)$

Retorna: `z[i]` = tamanho do maior prefixo comum entre `s` e `s[i..]`. Ex.: `"aaabaab"` → `[0,2,1,0,2,1,0]`.

```
vector<int> zFunction(const string& s) {
    int n = s.size();
    vector<int> z(n, 0);
    for (int i = 1, l = 0, r = 0; i < n; i++) {
        if (i < r) z[i] = min(r - i, z[i - l]);
        while (i + z[i] < n && s[z[i]] == s[i + z[i]])
            z[i]++;
        if (i + z[i] > r) l = i, r = i + z[i];
    }
    return z;
}
```

Busca de padrão igual ao KMP: calcule `z` de `p + '#' + t` e procure `z[i] == |p|`.

9.3 Hash polinomial — $O(1)$ por comparação

Retorna: `get(l, r)` = um número que identifica `s[l..r)`. Substrings iguais têm o mesmo hash, e diferentes colidem com probabilidade desprezível.

```
typedef unsigned long long ull;
const ull M = (1ULL << 61) - 1;

ull mulmod(ull a, ull b) {
    __uint128_t c = (__uint128_t)a * b;
    ull r = (ull)(c & M) + (ull)(c >> 61);
    return r >= M ? r - M : r;
}

const ull B = 911382323;   // base fixa

struct Hash {
    vector<ull> h, p;
    Hash(const string& s)
        : h(s.size() + 1, 0), p(s.size() + 1, 1) {
        for (size_t i = 0; i < s.size(); i++) {
            h[i+1] = (mulmod(h[i], B)
                      + (unsigned char)s[i]) % M;
            p[i+1] = mulmod(p[i], B);
        }
    }
    // hash de s[l..r)
    ull get(int l, int r) {
        return (h[r] + M - mulmod(h[l], p[r - l])) % M;
    }
};
```

Combine com busca binária para achar o LCP de dois sufixos em $O(\log n)$.

9.4 Manacher (palíndromos) — $O(n)$

Retorna: `d1[i]` = quantos palíndromos de tamanho ímpar têm centro em `i`, e `d2[i]` = quantos de tamanho par têm centro entre `i-1` e `i`. Ex.: `"abaaba"` → `d1 = [1,2,1,1,2,1]`, `d2 = [0,0,0,3,0,0]`.

```
int n = s.size();
vector<int> d1(n), d2(n);
for (int i = 0, l = 0, r = -1; i < n; i++) {
    int k = (i > r) ? 1 : min(d1[l + r - i], r - i + 1);
    while (i - k >= 0 && i + k < n && s[i-k] == s[i+k])
        k++;
    d1[i] = k--;
    if (i + k > r) l = i - k, r = i + k;
}
for (int i = 0, l = 0, r = -1; i < n; i++) {
    int k = (i > r) ? 0 : min(d2[l + r - i + 1], r - i + 1);
    while (i - k - 1 >= 0 && i + k < n
           && s[i-k-1] == s[i+k])
        k++;
    d2[i] = k--;
    if (i + k > r) l = i - k - 1, r = i + k;
}
// total de substrings palindromas: soma de d1 e d2
// maior palindromo: max(2*d1[i] - 1, 2*d2[i])
```

9.5 Trie — $O(|s|)$ por operação

Retorna: `count(s)` = quantas vezes `s` foi inserida, e `prefixCount(s)` = quantas palavras inseridas começam com `s`.

```
struct Trie {
    vector<array<int,26>> nxt;
    vector<int> endCnt, pass;
    Trie() { newNode(); }
    int newNode() {
        nxt.push_back({}); nxt.back().fill(-1);
        endCnt.push_back(0); pass.push_back(0);
        return nxt.size() - 1;
    }
    void insert(const string& s) {
        int u = 0; pass[u]++;
        for (char ch : s) {
            int c = ch - 'a';
            if (nxt[u][c] == -1) {
                int v = newNode(); nxt[u][c] = v;
            }
            u = nxt[u][c]; pass[u]++;
        }
        endCnt[u]++;
    }
    int walk(const string& s) {   // nó de s ou -1
        int u = 0;
        for (char ch : s) {
            u = nxt[u][ch - 'a'];
            if (u == -1) return -1;
        }
        return u;
    }
    int count(const string& s) {
        int u = walk(s); return u == -1 ? 0 : endCnt[u];
    }
    int prefixCount(const string& s) {
        int u = walk(s); return u == -1 ? 0 : pass[u];
    }
};
```

Para alfabeto diferente de `a`–`z`, ajuste o 26 e o `ch - 'a'`.